Block Operators

Anjali Kanvinde
akanvinde@umass.edu

Bhavana Anand
banand@umass.edu

Gomathy Dhanya Sankara Subramanian
gsankarasubr@umass.edu

Sameeksha Bhatia
sameeksharaj@umass.edu

I What General Problem Does the Project Address?

A growing share of useful data is unstructured, such as clinical notes, contracts, reviews, and support tickets, and the questions people want to ask of it turn on meaning, not exact column values. A relational `WHERE` clause can test `price > 100`, but it cannot test "this clause is unfair to the customer."

Semantic Operators

Semantic query processing systems like LOTUS [3] overcome this by letting a query be written similar to the relational way, with some steps answered by an LLM instead of being computed. These steps are called semantic operators. Applied to the contract case above: a semantic filter keeps contracts whose termination clause looks like it conflicts with policy, a semantic join matches each flagged contract to the internal policy clause it conflicts with, and a semantic map produces a one-line summary of the conflict. Similarly to their relational counterparts, these operators are composed into pipelines with the output of one feeding directly into the next.

Kalypso

Running a query like this means turning each of those steps into one or more LLM inference requests, executed on an inference engine such as vLLM [4]. Kalypso [1] is a relational LLM serving system that sits between the semantic query engine and that inference engine. A plain inference engine treats every request as independent and has no visibility into the query being run, so it can only reuse a model's cached internal state, or KV cache, for a shared prompt opportunistically, and that state is evicted as soon as memory runs short. Kalypso instead has access to the query plan itself, which operators exist, how their outputs feed one another, and which of their prompts share text, and uses that to schedule requests deliberately. It launches a dependent operator on a tuple while the upstream operator's cached state for that tuple is still resident, and manages GPU memory per pipeline stage so that reuse is preserved rather than left to chance. This memory-aware pipelining and scheduling has been reported to reduce the end-to-end query completion time by up to 4.57× over request-centric baselines.

Kalypso does not change the granularity at which rows become requests. Every operator built on it, similar to LOTUS, processes one tuple at a time. A filter over N rows is N separate requests, and a join issues one request per candidate pair, so a table of 500 contracts checked against 20 hypotheses is already 10,000 requests before any pipelining begins. Each request repays the same fixed cost of the system prompt and the instructions, a full prefill pass over both, for a single row's decision. Since LLM inference is already the dominant cost in these systems, that overhead scales directly with data size, and for joins, with the square of it.

Conventional database engines tune decisions like block size and join order as part of choosing an execution plan. No semantic query system tunes request granularity the same way.

Block Operators

A single request could instead carry a block of several rows, with the model returning a decision for each one in the same response, rather than being invoked once per row. For a filter, one prompt includes several rows at once, and the model returns a decision for each row in that same response, instead of one row per request. For a join, a block operator compares a block of left rows against a block of right rows in a single request and returns every matching pair it finds, instead of a separate request per candidate pair. The aim is that the fixed cost of the instructions and system prompt is amortized across many rows at once, and for joins specifically, the same left-hand row's context no longer has to be present for every candidate it is compared against. The trade-off is that a block is a harder single reasoning task than one row, with more structure for the model to get right in its response, and both can degrade accuracy as the block grows.

Our project will investigate how different block sizes affect execution time, memory use, throughput, KV-cache reuse, and accuracy, and determine when block-based execution performs better than processing tuples individually.

II Limitations of Existing Solutions

#	**Limitation**	**Example**	**Implication**
1	LLM operators often process one tuple at a time.	Filtering 100,000 rows could require 100,000 LLM calls.	Large datasets can lead to many separate requests.
2	The same instructions are repeated across requests.	A 200-token instruction repeated for 100,000 rows adds about 20 million repeated instruction tokens.	The model repeatedly processes the same prompt content.
3	Semantic joins may compare tuple pairs separately.	Joining 1,000 rows with 1,000 rows could require up to 1 million pairwise checks.	Joins can create a very large number of LLM calls.
4	vLLM can batch requests, but they are still separate LLM calls.	Several requests may run together, but each still has its own prompt.	Batching helps execution, but repeated prompt work still remains.

In Kalypso specifically, these costs are measured as follows: it caps predicate operators at an 8-token output, yet issues on the order of 7,000 oracle calls for BioDEX and 17,000 for ContractNLI. Checking 500 contracts against 20 hypotheses alone is 10,000 requests before any pipelining begins. Automatic prefix caching reuses a shared prompt prefix but doesn't remove the per-call overhead rows 2 and 4 describe, or change the fact that each row is still a separate invocation.

Packing multiple rows into one call has been tried before. Trummer's cost model batches candidate row-pairs into blocks specifically for the semantic join operator [2], but it targets a hosted, API-billed model with no local serving engine, KV cache, or query pipeline, so it doesn't apply to a system like Kalypso.

Kalypso has that local, pipelined infrastructure but no block-operator support inside it, in three concrete ways:

- Its Task abstraction wraps a single tuple and reserves and releases memory per task, with no representation of several tuples sharing one request.
- Its scheduler's starvation and saturation signals are a count of queued tasks, meaningful only when each task is one row.

- Its Cartesian-product operator compares one left tuple against the entire right table one row at a time, with no code path that ever forms or compares a block of rows on either side.

III Proposed Approach

Each stage of a Filter1 → Join → Filter2 pipeline runs with pipelined semantics, reusing Kalypso's existing stage/task abstraction, and each block-boundary decision is weighed against the cost of invalidating a reusable KV-cache prefix.

Core Algorithms

Block Filter, with targeted repair on partial failure:

1. Build one prompt containing the system prompt, all rows in the block and the predicate.
2. Send it as a single call to the model and parse the response into a per-row answer.
3. Identify any rows that came back without a usable answer (missing or malformed).
4. If some rows failed but not all: retry just the failed rows as a smaller block, repeating this same procedure on that subset.
5. If every row in the block failed: stop batching and answer each of those rows individually instead.
6. Once every row has a valid answer, return the rows that satisfied the predicate.

Pseudocode:

```
function BLOCK_FILTER(B, predicate, max_tokens):
    prompt = SYSTEM_PROMPT + FORMAT_ROWS(B) + predicate
    response = LLM_CALL(prompt, max_tokens)
    results = PARSE_STRUCTURED(response)

    bad = [r for r in B if r.id not in results or MALFORMED(results[r.id])]
    if bad and len(bad) < len(B):
        results.update(BLOCK_FILTER(bad, predicate, max_tokens))
    elif bad:
        for r in bad: results[r.id] = SINGLE_TUPLE_CALL(r, predicate)
    return [r for r in B if results[r.id] == True]
```

Block Join, with candidate sub-blocking on repair:

1. Build one prompt containing the left block, the right block and the join predicate.
2. Send it as a single call and parse the response into a set of claimed matching pairs.
3. Check every claimed pair against the row IDs actually present in the two blocks.
4. If all pairs are valid, accept them as the final matches.
5. If some pairs reference invalid IDs, isolate just the left and right rows involved in those bad pairs.
6. If that isolated set is small enough, check every possible pairing between them by hand.
7. Otherwise, run this same block-join procedure again on that smaller isolated set.
8. Combine the valid matches from the original response with whatever the repair step produces.

Pseudocode:

```
function BLOCK_JOIN(A_block, B_block, predicate, max_tokens):
    prompt = SYSTEM_PROMPT + FORMAT_ROWS(A_block,"left")
             + FORMAT_ROWS(B_block,"right") + predicate
    response = LLM_CALL(prompt, max_tokens)
    pairs = PARSE_STRUCTURED(response)
    valid, invalid_ids = VALIDATE_IDS(pairs, A_block, B_block)

    if invalid_ids:
        A_sub, B_sub = ISOLATE_AFFECTED(invalid_ids, A_block, B_block)
        if len(A_sub) * len(B_sub) <= SINGLETON_THRESHOLD:
            valid += EXHAUSTIVE_PAIRWISE_CALL(A_sub, B_sub, predicate)
        else:
            valid += BLOCK_JOIN(A_sub, B_sub, predicate, max_tokens)
    return valid
```

Tuple reliability and length filtering: Each tuple's appearances in `bad` (Block Filter) or `invalid_ids` (Block Join) are counted across every block it has been part of. A tuple that keeps triggering repair regardless of which other rows share its block is flagged unreliable and routed straight to singleton calls for the rest of the query, instead of being retried in another block. Rows far longer than the rest of a candidate block are pulled out before the first call too, using a simple token-length outlier check, since an oversized row is cheap to catch without ever invoking the model.

Block-Size Controller: Online Pipeline-Aware Hill Climbing. A candidate resize is rejected outright if it would push $\text{Accuracy}_{\text{empirical}}(\text{size})$ below a fixed accuracy floor set during calibration, before any of the cost-based comparisons below run. This avoids the failure mode where a resize with a large memory/latency win numerically outweighs a small accuracy loss and gets accepted anyway.

The floor itself is not a single number carried in from offline profiling. A small labeled calibration sample, the same kind already checked by hand for Milestone 1, is periodically re-run against the current block size during execution, so the floor tracks the workload actually running rather than the dataset it was profiled on. Drift between calibration passes is caught with a shadow check: a small fraction of completed blocks are spot-checked by re-answering a few of their rows with `SINGLE_TUPLE_CALL` and comparing against the block's own answer, and a rising disagreement rate is treated the same as a floor violation.

Block-Size Controller: Online Pipeline-Aware Hill Climbing.

1. Start from the block size the cost-model estimate produces for this stage.
2. After each block completes, check whether the current size is violating the accuracy floor. If so, revert to the last size known to be safe and stop; skip the rest of this round.
3. Otherwise, check memory pressure. If the stage is approaching its memory cap, shrink the block by one step.
4. Otherwise, check whether the downstream stage is starved for work while this stage still has tuples waiting. If so, shrink the block by one step.
5. Otherwise, check whether growth looks safe: the last block finished at or under the predicted time, the downstream queue has been growing rather than shrinking since, and this is the second consecutive completion to show that pattern. If all three hold, grow the block by one step.

6. If none of the above apply, leave the size unchanged this round.
7. If the resulting size means the block boundaries no longer match how the input was formed, reform them only if the memory/occupancy gained outweighs the KV-cache reuse that reforming would destroy.

Pseudocode:

```
B = LEVEL4_ESTIMATE(stage)
on each block completion for this stage:
    stats = COLLECT_STATS(stage, B)
    if VIOLATES_ACCURACY_FLOOR(B):
        B = LAST_KNOWN_SAFE(B); continue

    if MEMORY_PRESSURE(stats):
        B = SMALLER(B)
    elif DOWNSTREAM_STARVED(stats) and UPSTREAM_HAS_WORK(stats):
        B = SMALLER(B)
    elif NEARBY_LARGER_IMPROVES(stats):
        B = LARGER(B)
    else:
        pass
    if BOUNDARY_CHANGE_REQUIRED(B):
        if LOW_OCCUPANCY_STREAK(stage) >= K and BLOCKS_SINCE_LAST_REFORM(stage) >= N:
            REFORM_BLOCK_BOUNDARIES(B)
```

Signal	Definition	Data source
`MEMORY_PRESSURE(stats)`	`stage.used / stage.capacity` exceeds a fixed ratio	Existing: `KVMemoryManager._stage_used`/`_stage_capacity`
`UPSTREAM_HAS_WORK(stats)`	this stage's own waiting queue is non-empty (Kalypso's existing starvation check, on this stage)	Existing: `Stage.is_starving()`/`ready_count()`
`DOWNSTREAM_STARVED(stats)`	the next stage's waiting queue length has been below its low threshold for the last w completions	Read `next_stage.ready_count()` from this stage's controller, deliberately the opposite direction from Kalypso's own `is_starving`, which looks at a stage's own queue, not its downstream neighbor's; the two must not be conflated in code
`NEARBY_LARGER_IMPROVES(stats)`	the most recently completed block's wall-clock completion time was at or below the rolling average completion time for this stage at this block size over the last m completions, and `next_stage.ready_count()` trended up (not down) in the ticks after it completed	Requires per-block completion timing (not currently exposed by `CompletionResult`) plus a short rolling window over both completion time and `next_stage.ready_count()`

Signal	Definition	Data source
Reform guard	downstream queue length has stayed below its low threshold for K consecutive completions, and at least N blocks have passed since this stage's last reform	Existing: `next_stage.ready_count()` plus a per-stage reform counter, no new instrumentation required

The controller changes size by at most one step per completed block and requires two consecutive agreeing observations before acting on `NEARBY_LARGER_IMPROVES`, to avoid reacting to single-block measurement noise.

Strategies

The table below lays out the strategies considered for incremental implementation:

Level	Algorithm	Benefit	Limitation
1. Fixed tuple count	`blocks = [table[i:i+N] for i in range(0, len(table), N)]`	Trivial to implement, predictable call count	Ignores row length: same N yields wildly different prompt sizes, and for joins, unmanaged candidate counts
2. Fixed token budget	Greedily fill each block until $\sum$tuple_tokens $\geq T$, budget reserves space for expected output (via MapRatioEstimator, already in Kalypso) as well as input	Stabilizes prompt size and memory reservation regardless of row-length skew	Token budget $\neq$ reasoning difficulty for joins: cheap-token candidates still overload the model well before the budget is full
3. Trummer's cost model	$(k^*, j^*) = \arg\min \frac{\text{Cost}(k,j)}{k \cdot j}$ s.t. SystemPrompt $+ k \cdot \text{len}(A) + j \cdot \text{len}(B) \leq C$; expected output term $\sigma \cdot k \cdot j \cdot l_{out}$ estimated via Kalypso's existing MapRatioEstimator rather than a hand-tuned constant	Principled, closed-form, accounts for selectivity and the context-window limit	No accuracy term and no prefill/decode distinction: picks the token-cheapest block size even when it is far too large to answer reliably
4. Trummer + empirical awareness	$\text{Cost}'(k,j) = \mu_{\text{prefill}} \cdot \text{Prefill} + \mu_{\text{decode}} \cdot \text{Decode} - \text{ReuseDiscount} + \lambda \cdot \text{DownstreamDelay}$, minimized s.t. context window and $\text{Accuracy}_{\text{empirical}}(k,j) \geq \text{floor}$	Fixes (3)'s blind spots using measured system behavior, aware of pipeline position	Still a single size computed before execution: cannot react if real selectivity/fanout diverges from the profiled sample
5. Online pipeline-aware hill climbing	Probe one neighbor step from the Level-4 estimate per block completion, keep the move only if it measurably improves pipeline delivery (see signal table above), accuracy is a hard veto, never blended in	Only level that reacts to live drift in selectivity, fanout, memory pressure, and downstream delivery mid-query, using actual measured cost rather than a threshold trigger	Risk of chasing noise if step size/confirmation window is tuned too aggressively

IV Implementation Challenges

Representing Blocks Internally

- Currently each Task in Kalypso wraps one tuple and memory gets reserved and released per task. There is no way for multiple tuples to share one request's memory footprint yet.
- The scheduler's starvation and saturation checks just count queued tasks. That assumes one task is one row, which breaks once a task can be a whole block.

Building the Block Join

- The join operator currently compares one left tuple against the whole right table, row by row. There is no implementation that forms or compares a block on either side, so Block Join is a new operator: new prompt construction, new response parsing, and new logic to figure out which of the $k \times j$ pairs the model is actually claiming as a match.

Reforming Pipeline Boundaries

- Filter1 shrinks the tuples, the join expands them through fanout, and Filter2 shrinks them again. A block that is the right size going into one stage can end up too small or too big going into the next, so boundaries need to be reformed along the way.
- However, reforming a block eliminates the KV-cache reuse that blocking was supposed to give us, so reforms can't fire on every opportunity. The guard only allows a reform once downstream starvation has persisted for several completions in a row and enough blocks have passed since the last reform, so a single noisy tick can't trigger a boundary rewrite.
- Per-block completion timing still isn't tracked and needs to be added for the controller's other signals, but the reform guard itself only needs the existing downstream queue length plus a per-stage reform counter.

Handling Malformed Responses

- A block response can come back missing rows, with duplicate or wrong IDs, with one row's answer bleeding into another's, or missing a real join match because too many candidates were crammed into one prompt.
- The repair path deals with this by retrying the failed rows in a smaller sub-block, then falling back to one row at a time once that sub-block is small enough. That decision has to happen automatically, row by row, with nobody checking the output by hand.
- Flagging a tuple as unreliable needs a small persistent counter per tuple ID, tracked across every block it appears in for the life of the query, on top of the repair path itself.

Accuracy vs Speed

- Bigger blocks are cheaper per row, but the model has more to reason about at once, so a speedup can come at the cost of accuracy without anyone noticing. The accuracy floor is expected to catch that, assuming the accuracy estimate behind it is right, which the evaluation is meant to verify.
- The floor also needs a periodic recalibration pass against a small labeled sample and a low-rate shadow check against singleton calls, and both have to run without stalling the pipeline they are monitoring.

V Evaluation Criteria

Four metrics for every experiment are below:

- **End-to-end query latency**: wall-clock time from when the query starts to when the last output tuple is produced.
- **Number of LLM requests**: total count of LLM calls issued to answer the query, the thing block operators are trying to reduce.
- **Semantic accuracy**: how often a block operator's filter decision or join match agrees with the correct answer, checked against per-tuple execution as ground truth.
- **Peak memory usage**: the highest KV-cache / GPU memory occupancy reached at any point during execution.

Experiment	What it tests	Metrics applied
Block Filter scaling	Vary filter block size alone (e.g., 5, 10, 20, 40 rows per block), isolated from the rest of the pipeline	Latency, request count (should fall sharply), accuracy (checked for degradation), peak memory (per-block reservation size)
Block Join scaling	Vary both block dimensions k and j (e.g., 2, 5, 10 rows per side), since candidate count grows faster than in a filter	Same four metrics, with accuracy specifically checked against missed matches as $k \times j$ grows
Five-Level policy comparison	Run the same workloads under each of the five sizing strategies, confirming each level improves on the last and Level 5 matches Level 4's accuracy while beating it on the rest	Accuracy (must hold at or above the calibrated accuracy floor), latency and request count (should improve level over level), peak memory
Pipelined Filter1, Join, Filter2	Confirm the join starts before Filter1 finishes and Filter2 starts before the join finishes, compared against a materialized, non-pipelined run	End-to-end latency (primary comparison), peak memory (concurrent stages hold more state at once)
Block reforming on vs off	Repeat the pipelined run with boundary reforming disabled and enabled, under varying selectivity and fanout	Latency and peak memory (the tradeoff reforming is meant to improve), request count (reforming shouldn't increase this)
Cost model check	Compare the block size Trummer's model predicts as cost-optimal against the block size actually fastest under vLLM, same join workload	Latency (real-world outcome), request count (the token-cost proxy the model optimizes for)

VI Milestone Plan

Milestone 1 (Oct 7, progress discussion)

Team Member	Concept	Deliverable	Eval Metric
Teammate 1	Block Filter	Task abstraction extended so multiple tuple IDs can share one memory reservation	

Team Member	Concept	Deliverable	Eval Metric
		Block Filter runs standalone on a 20-row synthetic block; accuracy checked by hand against 10 known predicate answers, target 10 of 10 correct	Semantic accuracy
Teammate 2	Block Join	Block Join prompt construction and response parsing implemented	
		Runs standalone on a 5 by 5 synthetic block pair and returns correctly parsed matches, checked against a manual pairwise baseline, target full agreement	Semantic accuracy
Teammate 3	Controller/Scheduler	Scheduler's starvation and saturation signals redefined at block granularity	
		Levels 1 and 2 implemented and callable; request count on a 50-row sample measured against the per-tuple baseline, target at least 5 times fewer requests	Number of LLM requests
Teammate 4	Evaluation	Local vLLM instance running with the smallest viable model	
		Baseline per-tuple and vLLM-batching numbers collected on one workload (e.g. BioDEX subset): latency, request count, and peak memory logged and reproducible	Latency, requests, peak memory

Milestone 2 (Oct 21, progress discussion)

Team Member	Concept	Deliverable	Eval Metric
Teammate 1	Block Filter	Repair path implemented (recursive sub-blocking, singleton fallback)	
		Tested against injected malformed responses, target correct recovery on at least 90% of injected failures	Semantic accuracy
		Block Filter integrated as a real Kalypso stage	
		Per-tuple repair counter added, a tuple that triggers repair across at least 3 different block compositions is flagged unreliable and routed to singleton calls for the rest of the query	
Teammate 2	Block Join	Candidate sub-blocking repair implemented	
		Block Join integrated as a real Kalypso stage	
		Correctness validated on the 500 by 20 contract-style workload, target at least 95% agreement with an exhaustive pairwise baseline	Semantic accuracy

Team Member	Concept	Deliverable	Eval Metric
Teammate 3	Controller/Scheduler	Levels 3 and 4 implemented (Trummer's cost model, Trummer plus empirical awareness)	
		Per-block completion timing instrumentation added, plus a per-stage reform counter, target logged for 100% of completed blocks	
Teammate 4	Evaluation	Harness extended to run Block Filter and Block Join independently across a range of block sizes	
		All four metrics (latency, request count, accuracy, peak memory) logged per run	Latency, requests, accuracy, peak memory
		Target: at least 4 block sizes benchmarked per operator	

Milestone 3 (Nov 4, graded: fully functioning prototype)

Team Member	Concept	Deliverable	Eval Metric
Teammate 1	Block Filter	Block Filter fully integrated into the Filter1 to Join to Filter2 pipeline	
		Runs concurrently with the other stages, verified by overlapping wall-clock timestamps rather than sequential materialization	Latency
Teammate 2	Block Join	Block Join integrated into the same pipeline	
		Demonstrably begins consuming Filter1's output before Filter1 finishes, target measured start-before-finish overlap greater than 0 seconds, logged with timestamps	Latency
Teammate 3	Controller/Scheduler	Level 5 hill-climbing controller active in the pipeline, including the accuracy-floor veto and block-boundary reforming	
		At least one reform event logged during a demo run, with accuracy staying at or above the Level-4 floor throughout	Semantic accuracy
		Periodic recalibration and shadow check both active: floor re-verified against the labeled calibration sample at least once per demo run and a spot-check sample of completed blocks is re-answered with `SINGLE_TUPLE_CALL` and compared for disagreement	
Teammate 4	Evaluation	End-to-end demo ready: a query runs through all three concurrent stages and produces output	

Team Member	Concept	Deliverable	Eval Metric
		All four metrics logged automatically for the full run, this is the team's shared fully functioning prototype checkpoint, demonstrated together	Latency, requests, accuracy, peak memory

Milestone 4 (Nov 18, progress discussion)

Team Member	Concept	Deliverable	Eval Metric
Teammate 1	Block Filter	Block Filter scaling experiment complete: accuracy versus block size curve across at least 5 block sizes, written up	Semantic accuracy
Teammate 2	Block Join	Block Join scaling experiment complete: accuracy versus $k \times j$ curve across at least 5 block-size pairs, written up	Semantic accuracy
Teammate 3	Controller/Scheduler	Five-Level policy comparison and reforming on/off ablation complete	
		Level 5 shown to match Level 4 accuracy while improving latency or peak memory by a quantified margin, written up	Semantic accuracy, latency, peak memory
Teammate 4	Evaluation	Cost-model check complete: Trummer-predicted block size compared against the empirically fastest block size, with the measured latency gap quantified	Latency, requests
		All results consolidated into the final evaluation write-up	

References

[1] H. Son, M. A. Islam, H. G. Cao, H. Guan, M. Serafini. *Kalypso: Relational LLM Serving.* arXiv:2607.23815, 2026.

[2] I. Trummer. *Implementing Semantic Join Operators Efficiently.* arXiv:2510.08489, 2025.

[3] L. Patel et al. *LOTUS: Semantic Query Processing over Tables with LLMs.* 2024.

[4] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM).* SOSP 2023.